ATERA

Forward Deployed Engineer

Home Assignment

Introduction

This assignment evaluates how you solve real IT support problems by building automations inside Robin, Atera's AI service-desk agent, using scripts and APIs. It is a hands-on build exercise, not a writing exercise. You will work directly in a live Robin demo environment and make Robin resolve real ticket use-cases end to end.

We care about your judgment and problem-solving far more than polish. There is no single correct answer. Show us how you think.

Deliverables & Guidelines

- **Format:** This is a hands-on assignment. You build directly in the Robin demo environment described below.
- **Time budget:** 3 to 4 hours. Please do not spend more. Depth on the core task beats breadth. If you run low on time, cut scope on purpose and say why.
- **Turnaround:** Please return within 3 days of receiving this document.
- **Submit:** Leave your work in place in the demo environment, and send a short write-up (1 to 3 pages) or a 5 to 10 minute screen recording explaining what you built and why.
- **Ambiguity:** Where something is ambiguous or data is missing, state your assumption and keep moving. Do not wait for permission.
- **What comes next:** This is stage one. If you move forward, you will present and defend your solution to a customer stakeholder in a live follow-up session. Build with that conversation in mind. See "Next Stage" at the end of this document.

Demo Account Access

You will complete this assignment inside a shared Atera demo environment. Treat it like a customer environment: leave it tidy and name your work clearly so we can find it.

Platform	app.atera.com
Username	Please contact Michael Genkin via email <michael.genkin@atera.com> for login credentials only. All other communication should be with your hiring manager.

2FA	Required on first login (Authy, Microsoft Authenticator, or Google Authenticator).
API access	An API key for the Atera Public API is available in the environment under Admin > API. You will need it for Part 2.
Test device	A Windows test device with the Atera agent installed is available in the environment. Use it to run and validate your action and to demonstrate Robin resolving the ticket.

How Robin Works (2-minute primer)

Robin is Atera's AI IT service-desk agent. An end user opens a ticket ("I can't log in", "my laptop is slow", "new hire starts Monday") and Robin tries to resolve it end to end, with no technician. The building blocks you will use:

- **Action:** a single executable unit Robin can run. Two types matter here: a script action (PowerShell, Bash, or Python that runs on the end user's endpoint through the Atera agent), and a cloud / API action (an HTTP call to an external system such as Microsoft Entra ID, Okta, or Jira to do something off-device). Each action has parameters, a supported OS, a risk level, and returns output Robin reads.
- **Playbook:** an ordered workflow Robin follows to resolve one ticket category: classify, gather info, run action(s), verify, then respond or escalate. Playbooks have branches and guardrails.
- **Knowledge base and lexicon:** articles Robin cites when the answer is informational, plus a lexicon that maps the customer's own terms to real systems.
- **FRR:** First Resolution Rate, the metric that defines success. FRR = tickets auto-resolved by Robin (including via playbooks) / total Robin tickets. If a playbook does not move FRR, it is not working.

One thing to know before you start: Robin already ships a large action library. It includes endpoint scripts (disk and cache cleanup, print spooler, app installs, DNS flush) and cloud actions that reach external systems, for example a built-in Okta identity suite that resets passwords and MFA, enables and disables users, and adds or removes users from groups. This assignment is about building capabilities Robin does not already have. Do not rebuild what ships.

Every action Robin runs is autonomous. Nothing double-checks each run by hand. Design for that.

Part 1 – Solve Real Use-Cases with Robin (Build)

Background

Robin already covers a great deal out of the box: endpoint hygiene through script actions, and the core identity lifecycle through cloud actions (password reset, MFA reset, enable and disable user, group membership, against Okta). In a real deployment Robin also integrates with identity providers, ITSM

tools, and SaaS admin consoles, and a Forward Deployed Engineer builds those integrations often. You may reference that bigger picture in your thinking.

For this assignment, though, you will not have admin access to any external system such as Entra, Okta, Zoom, or ServiceNow. So the problem you solve must be one you can build and demonstrate entirely inside the Robin demo environment: a script-based action that runs on a test endpoint, orchestrated by a playbook, and if you want an external call, one to a public API that needs no private credentials. The problems below are chosen because Robin has no out-of-the-box action for them today, and because you can fully solve them with what the sandbox gives you. This is a test of creativity and technical skill within real constraints.

Your Task

First, search Robin's existing action library and confirm nothing already solves the problem you pick. Tell us what you searched for and what you found. Then choose ONE of the following problems and take it all the way end to end in the demo environment (pick a second only if you keep both tight). Depth on one beats shallow on several.

- Intelligent network triage and self-heal. A user reports intermittent or slow internet. Robin ships isolated actions like Flush DNS and SpeedTest, but nothing that diagnoses and fixes intelligently. Build an action that runs a layered check (adapter, driver, DHCP lease, gateway, DNS resolution, latency, packet loss), decides the most likely cause, applies safe fixes in order, re-tests, and returns a structured result. It should escalate rather than guess when it detects a hardware or upstream network fault.
- Software vulnerability check via a public API. A user asks whether anything on their machine is out of date or risky. Build an action that inventories installed software and versions, queries a free public vulnerability API that needs no credentials (for example OSV.dev), maps the findings, returns a prioritised report, and optionally remediates the top items through winget. This is your chance to show real API work without needing any enterprise tenant.
- Endpoint health and compliance scorecard. "Is my laptop healthy and compliant?" Robin has isolated checks (Bitlocker status, local admins) but no consolidated, scored posture check. Build an action that evaluates a set of posture items (disk encryption, firewall, antivirus present and current, pending reboots, screen-lock timeout, patch level), scores them, auto-fixes the safe ones, and returns a structured pass or fail report showing what it changed and what needs a person.
- Smart application repair for an app with no self-heal action today. "App X keeps crashing or will not open." Build a repair flow that detects the version, clears the right caches or profile data safely with no data loss, repairs or reinstalls through winget if needed, verifies the app launches, and reports the outcome. Pick an app that Robin does not already have a self-heal action for.

For your chosen problem:

- Build the Robin action(s) as script actions that run on a test endpoint. Return structured, parseable output Robin can act on. Make them idempotent and safe to run unattended.

- If your solution makes an external call, use a public API that needs no private credentials. Handle timeouts, errors, and rate limits, and do not hard-code anything you would be embarrassed to ship.
- Configure a Robin playbook that classifies the ticket, gathers what it needs, runs the action(s), verifies the result, and then either confirms resolution to the user or escalates to a human.
- Add the guardrails: what must be true before Robin acts, what it must never do, and the exact conditions under which it hands off to a person.
- Run it against the test device in the environment and show Robin resolving the ticket (screenshots or a short recording).

You do not need Entra, Okta, Zoom, or ServiceNow access for any of this. If it helps you explain your thinking, you may describe how you would extend the same pattern to those systems in production, but that is commentary, not a deliverable. The working build must run in the sandbox.

Required Deliverables

- Evidence that you checked Robin's existing library first: what you searched for, and why the existing actions do not cover your problem.
- The action(s) and playbook, built and left live in the demo environment. Tell us their names so we can find them.
- Your scripts (and any API calls), pasted into your write-up as well, with error handling visible.
- A short walkthrough: what you built, the guardrails you chose, and where Robin escalates instead of acting.

Evaluation Criteria

Criterion	What we are looking for
Checked before building	Do you confirm Robin has no existing action before building, rather than duplicating what ships?
Creativity and problem-solving	Do you turn a vague ticket into a smart, well-scoped solution within the sandbox constraints?
Technical execution	Are your scripts safe, idempotent, and parseable? If you call an API, is it done cleanly?
Guardrails and failure handling	Are verification, escalation, and safe-by-default behaviour designed in rather than bolted on?
Safety under autonomy	Would you trust this to run unattended on a real person's machine?

Part 2 – Automate with the Atera API (Build)

Background

Forward Deployed Engineers routinely script against the Atera Public API to set up, migrate, or measure things at scale. This part checks that you can work programmatically against a real API, not just in the UI.

Your Task

Using the demo environment's API key and the Atera Public API, write a small program (any language) that does ONE of the following:

- Pull tickets over a time window and compute a simple resolution-rate metric (for example, how many were resolved by Robin versus by a technician), or
- Bulk-create a set of sites/customers or agents from a small CSV, or
- Any equivalent task you believe an FDE would genuinely need. Tell us why you picked it.

Whatever you choose, it must handle the real world: authentication, pagination, error handling including rate limits, idempotency where it matters, and clear output. Where the demo data is thin, use sample data and say so.

Required Deliverables

- The code, runnable, with brief run instructions.
- Sample output.
- Three or four lines on what you would harden before pointing this at a real customer.

Evaluation Criteria

Criterion	What we are looking for
API fluency	Correct auth, endpoints, and pagination against a real API.
Robustness	Handles errors, rate limits, and partial failure without falling over.
Judgment	Chooses a task that reflects real FDE work and explains the choice.
Communication	Clear output and an honest note on what is not yet production-ready.

Part 3 – Engineering Judgment (Written)

Short answers, a few sentences each. This should take about 30 minutes.

- **1:** Give a use-case where Robin should NOT auto-resolve and should escalate to a human instead. Explain why, and how you would encode that boundary so Robin respects it every time.
- **2:** You ship an action to a customer and, two weeks later, FRR for that ticket type has not moved at all. Walk through how you diagnose why.

- **3:** One of your scripts could be destructive if a parameter is wrong or an endpoint is in an unexpected state. In an autonomous system with no human in the loop, how do you make it safe?

Next Stage: Presenting to the Customer

If your submission moves forward, the next stage is a live session where you present and defend your solution to a customer stakeholder, played by our panel. Assume this is the customer's IT lead: technical enough to follow you, but focused on risk, trust, and outcomes rather than code. This is a Forward Deployed Engineer role, so how you carry a customer through your thinking matters as much as the build itself.

Expect to:

- Walk the customer through the problem, what you built, and how Robin now resolves the use-case end to end.
- Explain your guardrails in plain language: how you keep an autonomous system safe when it is touching their accounts, access, and licenses.
- Handle honest pushback from a customer who is cautious about letting AI make changes in their environment.
- Tie the work to outcomes the customer cares about: faster resolution, fewer tickets reaching a technician, and a measurable move in FRR.

Build with that conversation in mind. You do not need to prepare slides now. The write-up or recording you submit should be something you could comfortably talk a customer through.

Submission & Good Luck

Leave your actions and playbook in the demo environment, and send your write-up (or recording) plus your Part 2 code to your hiring contact within the agreed timeframe.

A strong submission will:

- Solve the use-case end to end, not just the happy path.
- Show real safety instinct: verification, least privilege, and clear escalation.
- Reuse building blocks (for example, an action built once and called from a playbook).
- Be honest about what was cut for time and what you would do next.

Good luck. We are looking forward to seeing how you think.